Technical Report

Repair-Help Platform – „Marburg Repair Hub“

A web platform for repair cafés and repair services in Marburg

Student project according to the „Guideline for advanced internship / project work“
Interdisciplinary Module: Creativity and Innovation in Regional Ecosystems (CIM Hub)
Philipps-Universität Marburg

Winter Term 2025/26 and Summer Term 2026

Team
Ali Heydari (M.Sc. Computer Science) – both semesters
Sayed Amirabbas Modares Motlagh (M.Sc. Computer Science) – both semesters
Lars Lauer, Florian Rademaker (B.Sc. Business Informatics) – winter term 2025/26
Saiyed Mohammad Sanan, Shaikh Mohmed Shoban, Sakher Ahmed Elshteiat – summer term 2026

Version 1.0 – October 2026

Contents

1 Planning ... 3
1.1 (A1) Brief description ... 3
1.2 (B1) Development process ... 3
1.3 (B2) Team ... 4
1.4 (B3) Risk management ... 5
1.5 (B4) Timetable ... 5
2 Requirements analysis ... 7
2.1 (C1) Definition of the target system ... 7
2.2 (C2) Functional requirements ... 7
2.3 (C3) Non-functional requirements ... 8
3 Design ... 10
3.1 (D1) Rough design (architecture) ... 10
3.2 (D2) Technologies ... 11
3.3 (D3) Detailed design ... 11
4 Quality assurance ... 14
4.1 (E1) Test plan ... 14
4.2 (E2) Test protocol ... 15
5 Final report ... 17
5.1 (A2) Summary ... 17
5.2 (F1) User documentation ... 17
5.3 (F2) Developer documentation ... 19
5.4 (A3) Field report ... 21

1 Planning

1.1 (A1) Brief description

The Repair-Help Platform (public name „Marburg Repair Hub“ / „Marburg Reparatur-Hilfe“) is a web application that collects the repair offers in and around Marburg – community repair cafés, self-help bike workshops and professional repair shops – and presents them on one website. Citizens can see all locations on a map, search for the device they want to have repaired, filter by service category and read the details of each location (address, opening hours, pricing, accessibility, website). The platform is an information hub: it does not carry out repairs, offers no booking and links to the official channels of the providers.

General conditions:

- **Embedding:** sub-project of the „Reparatur-Hilfe“ initiative in the interdisciplinary module „Creativity and Innovation in Regional Ecosystems“ (CIM Hub). A parallel student group („Reparatur-Hilfe“ team) is responsible for stakeholder contact and data collection; our group builds the website.
- **Client / supervision:** CIM Hub of Philipps-Universität Marburg (module lecturers, incl. the Software Engineering group, FB 12), with consultation of the initiative’s representatives.
- **Time frame:** two semesters – winter term 2025/26 (concept and static prototype) and summer term 2026 (dynamic web application, brand identity). Publication is planned for a following semester.
- **Special requirements:** a stakeholder survey in the first semester showed that the repair cafés have no capacity to maintain a website. Therefore all content must be maintainable centrally by one administrator without programming knowledge.
- **Quality requirements:** low operating cost and simple hosting (no complex infrastructure), German and English interface, usable on mobile phones, easy to take over by new student teams every semester.

1.2 (B1) Development process

We used an iterative and incremental process. Because the content of the platform depends on data that is collected by another group, we oriented the work on the phases of CRISP-DM (Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, Deployment) and went through them several times: the requirements were reviewed with the stakeholders (Business Understanding), the collected location data was structured into the data model (Data Understanding/Preparation), features were implemented (Modeling, here: implementation), tested (Evaluation), and the results fed back into the next iteration. Deployment has not taken place yet. In the first semester, the process was closer to a classic requirements-first approach (research, requirements specification, mock-up, prototype). In the second semester we worked in short iterations between the milestones M1–M5 (see 1.5).

Practices:

- Requirements as a prioritised feature list („necessary” / „optional”) derived from research, the stakeholder survey and consultations; at the end of the summer term a brainstorming produced the backlog for the next semester.
- Fixed responsibilities per component in the second semester (see 1.3); decisions in regular (mostly online) team meetings and in consultation with the CIM Hub, the representatives and the „Reparatur-Hilfe“ team.

- Incremental development: static HTML/CSS/JS prototype first, then conversion into a Flask application with a database while keeping the existing design.
- Manual testing during development; at the end of the summer term the team additionally ran automated unit/integration and GUI tests (chapter 4).
- Generative AI was used only as a supporting tool. The generated code was reviewed, adapted and tested by the team members.

Documents created: requirements specification and mock-up (winter term), group semester reports, this technical report, the README file in the code folder, the brand book „Repair Café Hub Marburg – Corporate Identity & Brand Guidelines“ (v1.0, June 2026) and the logo files.

Table 1: Languages and CASE tools

Area	Tool / decision
Programming languages	Python 3 (backend), HTML5 / CSS3 / JavaScript (frontend), SQL (SQLite)
IDE	JetBrains PyCharm (project files in the .idea folder); any editor can be used
Modeling tools	Diagrams in this report: Mermaid and Matplotlib (generated from the code)
Test framework	pytest 9 with the Flask test client; Playwright 1.56 (Chromium) for GUI tests
Test coverage	coverage.py 7.16 (statement coverage of app.py)
Continuous integration	not used so far (recommended for next semester, see 5.4)
Quality assurance tools	none automated; code reviews in the team
Project management / communication	WhatsApp group, regular online meetings; shared project folder (Google Drive in the winter term)
Configuration management	no version control system; versions were kept as separate folders (website → website2 → web2/marburg). Introducing Git is the first recommendation for the next team.
AI support	generative AI assistant used only as a supporting tool (results reviewed and tested by the team)

1.3 (B2) Team

Table 2: Team members and responsibilities

Member	Program	Semester(s)	Responsibilities
Ali Heydari	M.Sc. Computer Science	WiSe 25/26, SoSe 26	WiSe: architecture of the prototype, interactive map, CO_2 section. SoSe: frontend (templates, map, search/filter, details panel, language switch, brand implementation)
Sayed Amirabbas Modares Motlagh	M.Sc. Computer Science	WiSe 25/26, SoSe 26	WiSe: technical implementation of the prototype, data structure. SoSe: logo and brand book (visual identity, colour system, design rules for the website); author of design-related documentation
Lars Lauer	B.Sc. Business Informatics	WiSe 25/26	requirements analysis, interface concept, documentation and reports
Florian Rademaker	B.Sc. Business Informatics	WiSe 25/26	requirements analysis, interface concept, documentation and reports
Saiyed Mohammad Sanan	M.Sc. Computer Science	SoSe 26	backend (Flask, database), testing, documentation
Shaikh Mohmed Shoban	M.Sc. Computer Science	SoSe 26	backend (Flask, database), testing, documentation
Sakher Ahmed Elshteiat	M.Sc. Computer Science	SoSe 26	backend (Flask, database), testing, documentation

Roles were fixed per component in the summer term (frontend, backend/testing/documentation, brand identity); there was no formal Product Owner. Requirements were agreed with the CIM Hub and the „Reparatur-Hilfe“ team, who acted as the client side. Minutes were not kept systematically; the guideline’s recommendation of written minutes with a rotating minute-taker should be adopted in the next semester.

1.4 (B3) Risk management

Table 3: Risks, prioritised by probability and impact

#	Risk	Prob.	Impact	Strategy (avoid / minimise / emergency plan)	What happened
R1	Data about repair locations is late or incomplete (depends on another group and on stakeholders)	high	high	Minimise: develop with seed data, make all content editable in the admin panel so data can be added later	Occurred in both semesters; strategy worked, 13 locations integrated
R2	Repair cafés do not want / cannot maintain their own data	high	high	Avoid: central maintenance by one admin instead of café accounts	Confirmed by survey; scope reduced to an information hub
R3	Team change between semesters (knowledge loss)	high	medium	Minimise: README, reports, onboarding session, simple technology stack	3 new members in SoSe; onboarding took time at the start
R4	Availability of members (exams, other courses)	medium	medium	Minimise: online meetings, clear responsibilities	Some scheduling problems, no major delay
R5	External services (Leaflet CDN, Esri map tiles) change or are unavailable	low	high	Minimise: pin library version, host Leaflet locally; emergency: switch tile provider	Not yet mitigated (Leaflet URL is unversioned)
R6	Security of the admin panel	medium	high	Minimise: password outside the code, HTTPS, CSRF protection before going online	Open – must be solved before deployment

1.5 (B4) Timetable

The project followed the semester schedule of the module (topic presentation, interim presentation, final presentation). The following table shows the milestones of both semesters and whether the planning targets were achieved.

Table 4: Milestones and achievement of planning targets

Milestone	Planned	Content	Status
W1 Research & team setup	WiSe, beginning	team formation, analysis of comparable platforms, choice of tools	Achieved
W2 Requirements & mock-up	WiSe, first half	requirements specification v1, first mock-up, technical models	Achieved
W3 Static prototype	WiSe, second half	main page, Leaflet map, category cards, CO_2 section, initial data structure	Achieved
W4 Integrate survey results	end of WiSe	survey showed cafés cannot maintain a site → scope reduced to information hub	Achieved
M1 Kick-off & onboarding	SoSe, start	onboarding of 3 new members, goals for the semester	Achieved
M2 Technical basis	SoSe, first half	conversion to Flask + SQLite + Jinja2	Achieved
M3 Features & real data	SoSe, interim presentation	search/filter, admin panel, bilingual UI, 13 real locations	Achieved

Milestone	Planned	Content	Status
M4 Logo & brand book	SoSe, second half	logo, brand book v1.0 (June 2026)	Achieved
M5 Brand applied, testing, documentation	SoSe, end	colours/logo in website, testing, README, reports	Achieved
Deployment (go-live)	originally SoSe	hosting, e-mail, legal pages	Not achieved – moved to next semester

Effort was not recorded per task; for the next semester we recommend recording the actual effort in a simple spreadsheet so that it can be compared with the estimate, as required by the guideline. Short-term plan for the next semester: (1) Git repository and CI with the tests from chapter 4, (2) fixing the four failed tests, (3) deployment preparation (security, e-mail, legal pages, image optimisation), (4) backlog items listed in 5.1.

2 Requirements analysis

2.1 (C1) Definition of the target system

Table 5: Target system

Side	Hardware	Software
Server	any small server or virtual machine (1 CPU, 512 MB RAM are sufficient); ~100 MB disk incl. images	Linux, Windows or macOS; Python ≥ 3.9; Flask ≥ 3.0 (tested on Windows with Python 3.14, Flask 3.1); SQLite 3 (part of Python); for production a WSGI server (e.g. Gunicorn) behind a web server with HTTPS
Client	desktop, tablet or smartphone (from 375 px width)	current web browser with JavaScript (Chrome, Firefox, Edge, Safari); internet access for the Leaflet library and map tiles

2.2 (C2) Functional requirements

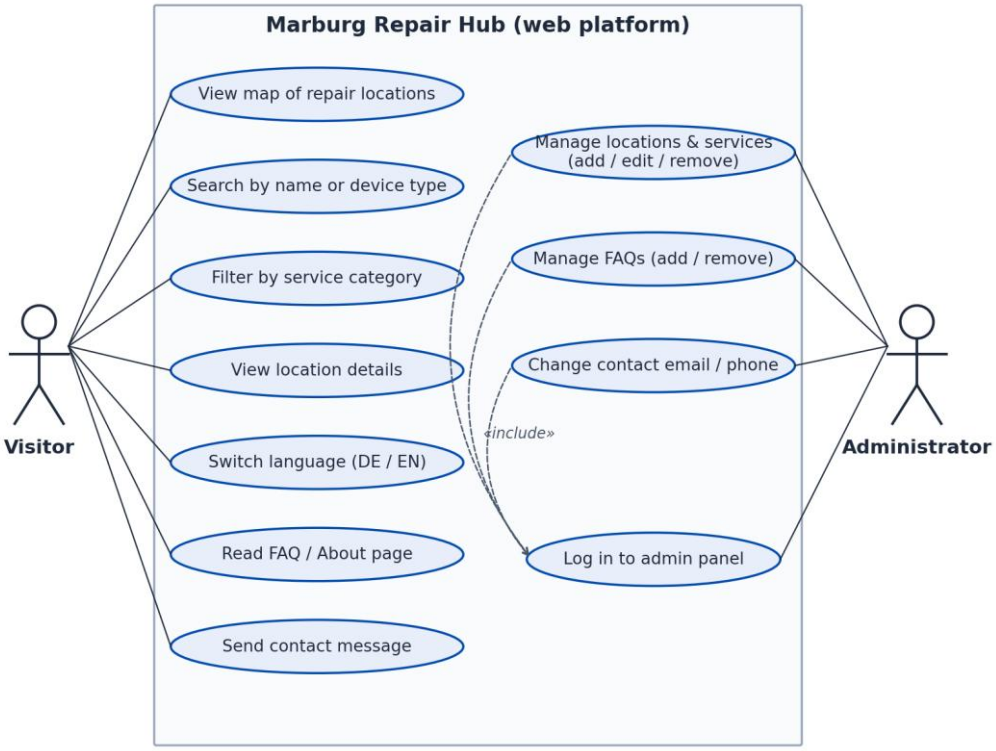


Figure 1: Use case diagram of the platform

The requirements were collected in the winter term (research of comparable platforms, requirements specification, stakeholder survey) and extended in the summer term. The table lists them with priority and implementation status; the last column refers to the test cases in chapter 4.

Table 6: Functional requirements

ID	Requirement	Prio	Sem.	Status	Tests
F01	Show all repair locations on an interactive map (markers) and in a list	necessary	WiSe	Implemented	T03, G01
F02	Show details of a location: address, hours, services, pricing, accessibility, notes, website	necessary	WiSe	Implemented	G07
F03	Repair category cards (phones, laptops, audio, bikes,	necessary	WiSe	Implemented	G06

ID	Requirement	Prio	Sem.	Status	Tests
	household, tools)				
F04	Environmental impact section (CO_2 saved by repairing)	optional	WiSe	Implemented	T02
F05	Live search by location name or device/service type (EN + DE terms)	necessary	SoSe	Partly (no tolerance for spelling variants)	G02–G05, G08
F06	Filter locations by clicking a category; click again to reset	necessary	SoSe	Implemented	G06
F07	Default list of all locations before any search/filter	necessary	SoSe	Implemented	T03, G01
F08	German and English user interface, switchable in the header	necessary	SoSe	Implemented	T05, T06, G09
F09	Many-to-many relation: a location can offer several services	necessary	SoSe	Implemented	T04, T15
F10	About, FAQ and contact pages	necessary	SoSe	Implemented	T02
F11	Contact form stores messages; receiver address configurable	necessary	SoSe	Partly (no e-mail sending, no server validation)	T07, T22, G10
F12	Admin panel reachable only via /admin, password protected	necessary	SoSe	Implemented	T09–T14
F13	Admin: add, edit and remove locations incl. services and coordinates	necessary	SoSe	Implemented	T15–T17, T20, G11
F14	Admin: add and remove bilingual FAQs	necessary	SoSe	Implemented	T18
F15	Admin: change contact e-mail and phone shown in the footer	necessary	SoSe	Implemented	T19
F16	Admin: add/remove all footer data (social links, legal links)	optional	backlog	Open	–
F17	Search that also finds similar entries (e.g. „lap top“ / „laptop“)	necessary	backlog	Open	G05
F18	Repair cafés can state which spare parts they have	optional	backlog	Open	–
F19	Check feasibility of buy/sell interface and login for cafés; eBay-like ideas	optional	backlog	Open	–

2.3 (C3) Non-functional requirements

The non-functional requirements were formulated so that they can be checked. The column „Result“ contains the values measured for this report (local measurement with the Flask test client and Chromium, see chapter 4).

Table 7: Non-functional requirements with measured results

ID	Category	Requirement (measurable)	Result	Status
N01	Performance	Server response time for /, /faq, /api/cafes < 200 ms (median, local)	1.6 ms / 0.9 ms / 0.9 ms	Met
N02	Performance	Total size of images loaded by the home page < 3 MB	≈ 84 MB (11 photos, up to 24 MB each)	Not met
N03	Usability	No horizontal scrolling on a 375 px wide phone screen	0 px overflow (G12)	Met
N04	Usability	100 % of interface texts available in German and English	94 / 94 text keys in both languages	Met
N05	Usability / brand	Colours follow the brand book (#004AAD, #FFD700, neutrals); gold never as text on white	palette used in style.css	Met
N06	Maintainability	All content (locations, services, FAQs, contact data) changeable without code	via admin panel; footer links and texts still in code	Partly
N07	Maintainability	New developer can start the app with ≤ 2 commands	pip install -r requirements.txt; python app.py	Met
N08	Reliability	Invalid admin input does not crash the application	invalid coordinates handled (T20); empty names accepted (T21)	Partly
N09	Portability	Runs on Linux servers (case-sensitive file	2 image names differ in case (T23)	Not met

ID	Category	Requirement (measurable)	Result	Status
		system)		
N10	Security	Admin password and secret key not stored in source code; protection against CSRF	both hard-coded; no CSRF tokens	Not met
N11	Code quality	Statement coverage of app.py by automated tests ≥ 80 %	98 %	Met
N12	Code quality	Every module and public function has a comment (guideline, chap. 3)	module comment yes; 3 of 26 functions with docstring	Not met
N13	Documentation	README with installation and admin instructions; technical report	available	Met

Technical requirements: Python/Flask and SQLite were required to keep hosting simple; the existing HTML/CSS design of the prototype had to be kept and integrated with Jinja2 templates; React or other large frontend frameworks were deliberately not used.

3 Design

Because we worked iteratively, the architecture was not fully designed in advance. It is documented here as implemented at the end of the summer term (as recommended by the guideline for agile projects).

3.1 (D1) Rough design (architecture)

In the winter term the platform was a static website: HTML pages, one CSS file and one JavaScript file (script.js) in which the 13 locations were stored as a hard-coded JavaScript array. The admin page of the prototype only showed forms; changes were not saved. In the summer term we converted it into a classic server-side web application with three tiers (Figure 2):

- **Presentation (client):** HTML pages rendered by Jinja2, style.css with the brand colours, script.js for the interactive map, search and filter, and the Leaflet library for the map.
- **Application (server):** one Flask application (app.py) with public routes, admin routes, a JSON API, the language handling (i18n dictionary + session) and data-access helper functions.
- **Data:** an SQLite database file (repair_hub.db) created from schema.sql and filled with seed data on the first start.

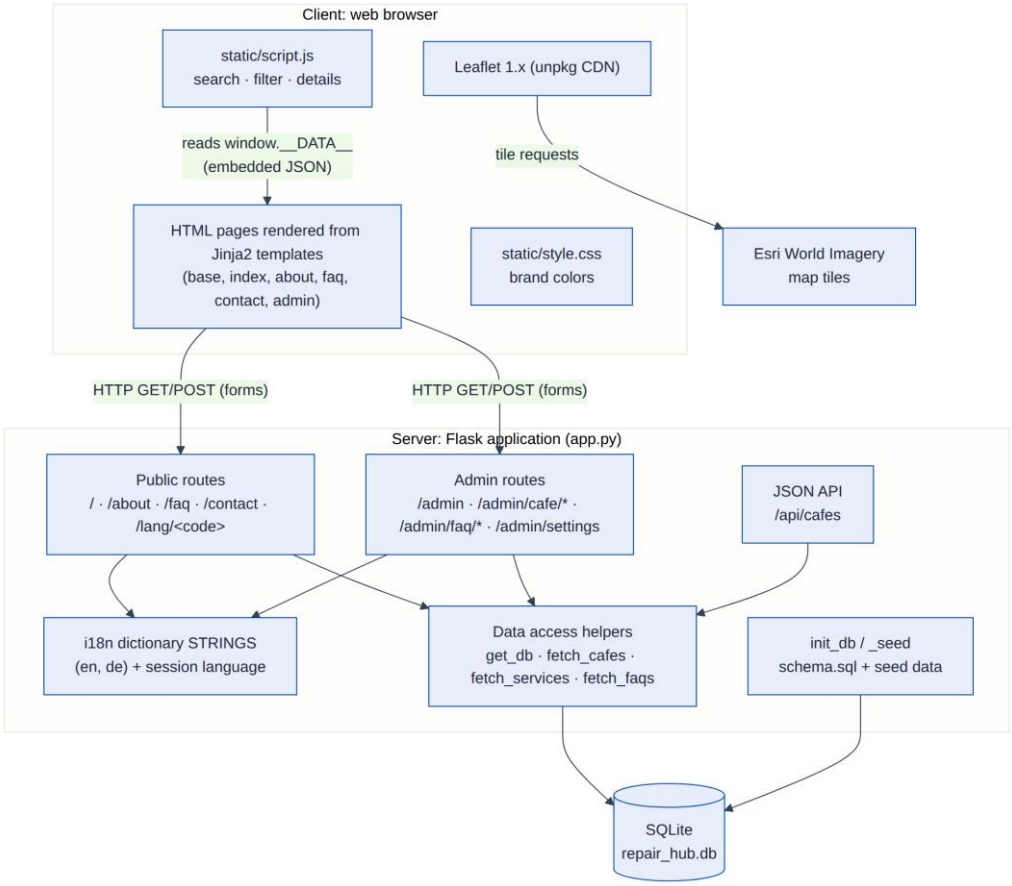


Figure 2: Architecture: components and interfaces

Interfaces between the components:

Table 8: Interfaces

Provided by	Interface	Used by
Flask public routes	HTTP GET /, /about, /faq, /contact, /lang/<code>; POST /contact (form)	browser

Provided by	Interface	Used by
Flask admin routes	GET/POST /admin (login), GET /admin/logout; POST /admin/settings, /admin/faq/add, /admin/faq/delete/<id>, /admin/cafe/add, /admin/cafe/edit/<id>, /admin/cafe/delete/<id>	admin in browser (HTML forms)
Flask JSON API	GET /api/cafes → list of locations with services (slugs, EN/DE names)	debugging, future clients
index.html	window.__DATA__ = {lang, cafes, strings} (JSON embedded in the page)	script.js
Data-access helpers	get_db(), fetch_cafes(), fetch_services(), fetch_faqs(), get_settings()	all routes
Leaflet / Esri	JavaScript map API (unpkg CDN), map tiles (Esri World Imagery)	script.js

Architectural patterns and strategies: server-side rendering with a template engine (Model–View–Controller in a simple form: SQLite tables as model, Jinja2 templates as view, Flask route functions as controller); Post/Redirect/Get after every form submission; a request-scoped database connection (Flask g object); progressive enhancement – the list of locations is rendered on the server and also works without JavaScript, while search, filter and map are added by script.js. Conventions: Python code follows PEP 8 naming (snake_case), route functions are named after their URL, template keys for texts use prefixes (nav_, c_ for contact, a_ for admin, d_ for details), and services are referenced by a stable slug (e.g. „bike“) that is also used as CSS class.

3.2 (D2) Technologies

Table 9: Technologies and libraries

Technology	Version	Purpose	Source / dependencies
Python	≥ 3.9 (tested 3.14)	backend language	python.org
Flask	≥ 3.0 (tested 3.1)	web framework, routing, sessions, flash messages	pypi.org/project/Flask; brings Werkzeug 3.1, Jinja2 3.1, itsdangerous, click, blinker
Jinja2	3.1 (via Flask)	HTML templates	dependency of Flask
SQLite	3.x (sqlite3 module)	embedded database	part of the Python standard library
Leaflet	not pinned (latest from unpkg)	interactive map, markers	https://unpkg.com/leaflet/dist/ – should be pinned, e.g. 1.9.4
Esri World Imagery	web service	map tiles (satellite images)	server.arcgisonline.com
HTML5, CSS3, JavaScript (ES6)	–	frontend	no build step, no frontend framework
pytest / coverage.py / Playwright	9.1 / 7.16 / 1.56	automated tests (development only)	pypi.org

3.3 (D3) Detailed design

The backend is written in a procedural style (functions, no own classes), as it is common for small Flask applications. To still give an overview in UML, Figure 3 shows the modules as classes with stereotypes: the module app.py with its helper functions, the two groups of route functions, the Jinja2 templates and the frontend script.

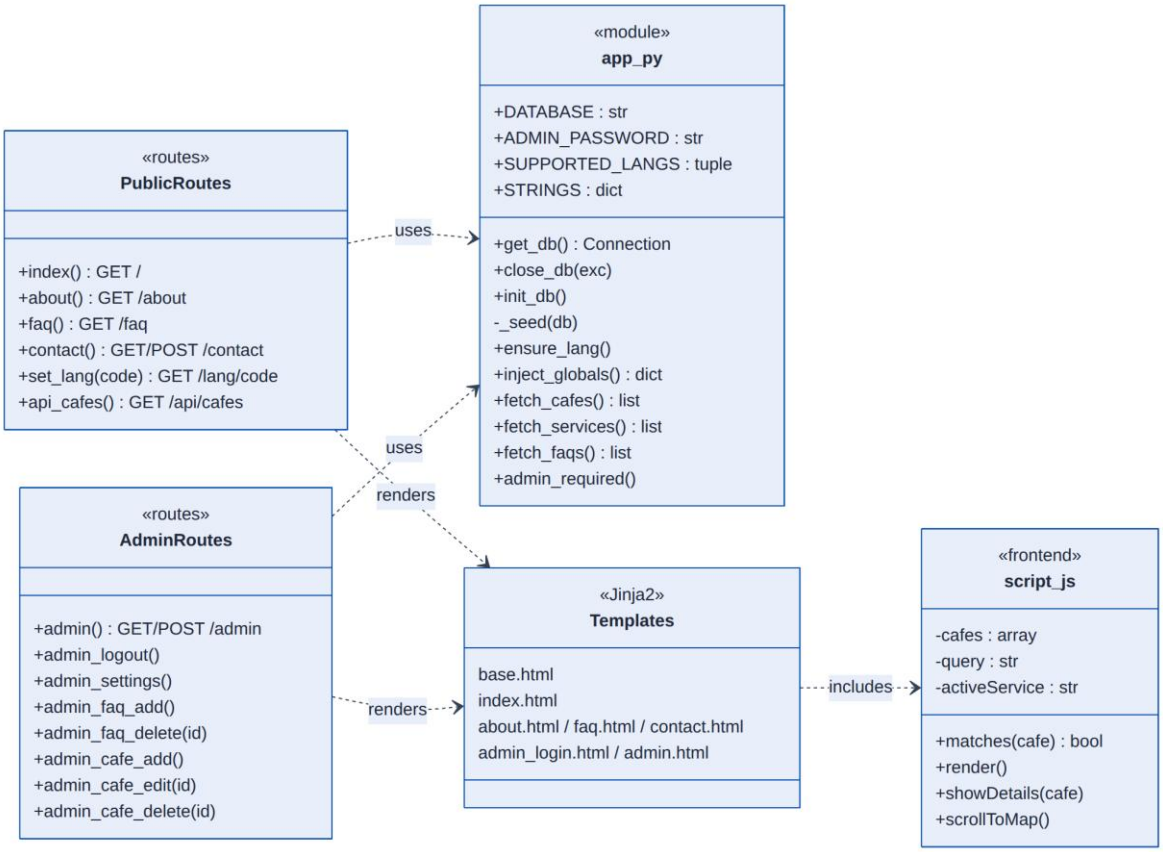


Figure 3: Module/class view of the implementation

Data model

The database has six tables (Figure 4). The relation between locations (cafes) and services is many-to-many and is stored in the join table cafe_services with foreign keys (ON DELETE CASCADE; foreign keys are activated with PRAGMA foreign_keys = ON for every connection). Texts that are shown to users are stored in both languages (name_en/name_de, question_en/question_de …). settings is a key-value table for the contact e-mail and phone; messages stores the submissions of the contact form.

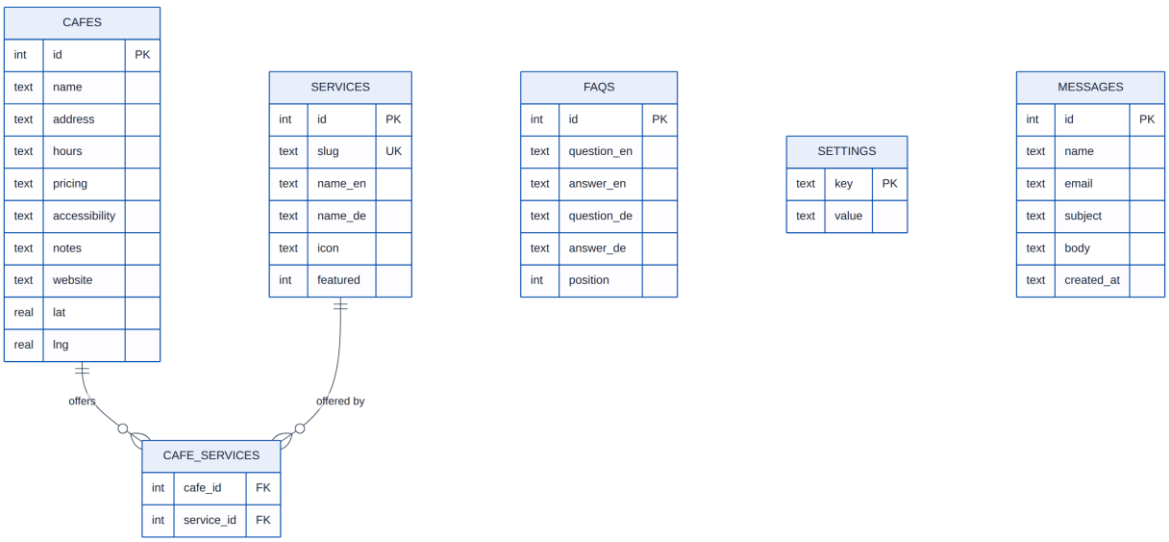


Figure 4: Data model (entity-relationship diagram)

Behaviour

Figure 5 shows the client-side search and filter logic of script.js. The function render() is called after every user action; a location is visible if it matches the active service category and if the search text is contained in its name or in one of its English or German service names.

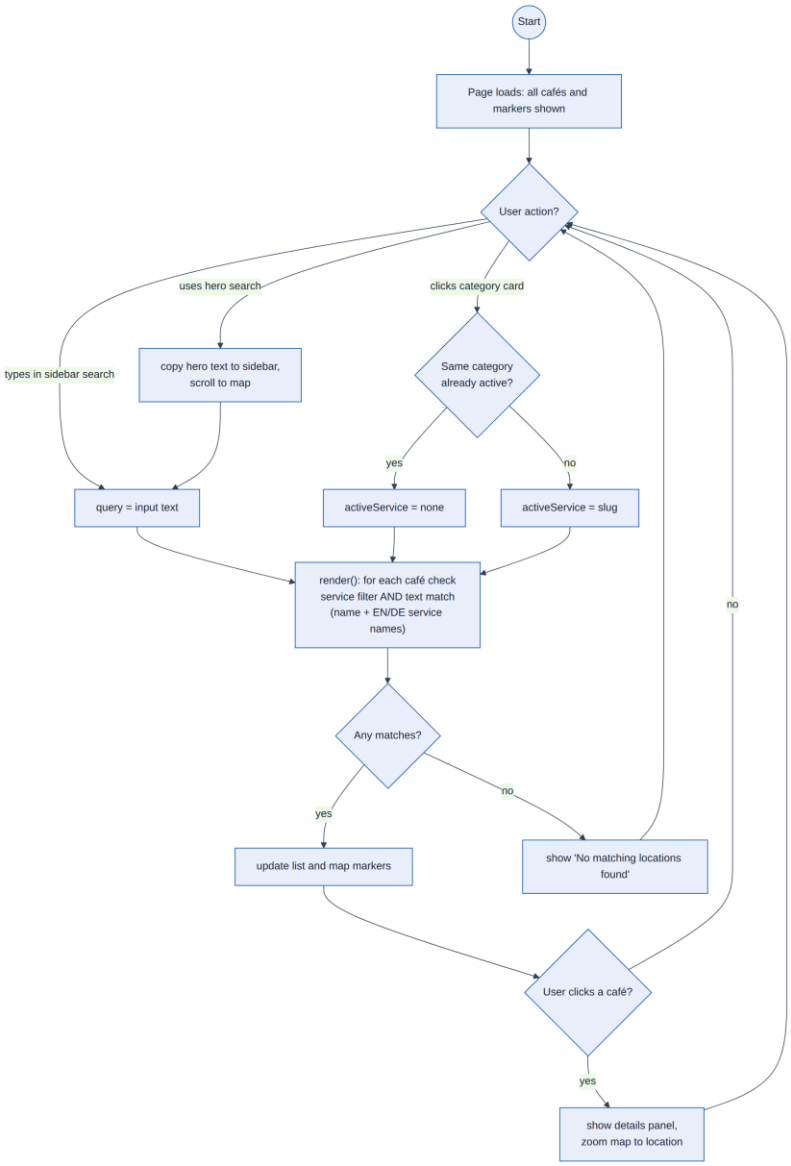


Figure 5: Activity diagram: search and filter on the home page

Responsibilities of the developers

Frontend (templates, script.js, style.css, brand implementation): Ali Heydari. Backend (app.py, schema.sql, seed data, admin routes) and tests: Saiyed Mohammad Sanan, Shaikh Mohmed Shoban, Sakher Ahmed Elshteiat. Visual identity (logo, colour system, brand rules used by the frontend): Sayed Amirabbas Modares Motlagh. The building blocks D4 (evidence idea) and D5 (algorithm design) of the guideline are not relevant for this project, because it contains no algorithms that need a formal proof or special algorithm design.

4 Quality assurance

4.1 (E1) Test plan

During development the website was tested manually by clicking through the pages in the browser. In addition, the team ran automated tests at the end of the summer term, so that the results are reproducible and can be run again by the next team (folder tests/ with conftest.py, test_backend.py and test_gui.py). Every test starts the application with a fresh copy of the code and a newly created and seeded database, so the tests are independent of each other and do not change the real database.

Table 10: Test methods

Test method	Tool	Used for
Unit / integration tests	pytest + Flask test client	routes, database seeding, admin authentication, create/edit/delete operations, language handling, input handling, static assets
GUI / system tests	pytest + Playwright (Chromium, headless)	real server and browser: map markers, live search, category filter, details panel, language switch, contact form, admin workflow, mobile layout
Performance measurement	Flask test client, 50 requests per URL	server response times (N01)
Manual review	–	code comments, image sizes, security settings (N02, N10, N12)

Coverage targets: ≥ 80 % statement coverage of the backend (app.py), and 100 % specification coverage of the „necessary“ functional requirements, i.e. every necessary requirement in Table 5 has at least one test. We consider this sufficient because the backend is small and consists mostly of straightforward database operations; the important risks lie in the interaction of backend, database and browser, which is covered by the GUI tests. The frontend JavaScript is covered functionally by the GUI tests (no line coverage measured). During the GUI tests the external map tiles are replaced by a blank image, so the results do not depend on the map service; Leaflet is loaded normally from the internet.

Test cases:

Table 11: Test cases

ID	Test case (expected result)	Type
T01	First start creates the schema and seeds 13 locations, 8 services, 3 FAQs, 31 service links	integration
T02	All public pages (/, /about, /faq, /contact) return HTTP 200	integration
T03	Home page lists all locations by default	integration
T04	/api/cafes returns all locations with their services	integration
T05	Language switch to German shows German texts; English is default	integration
T06	Unsupported language code (fr) is ignored	integration
T07	Contact form stores the message and shows a confirmation	integration
T08	Unknown URL returns 404	integration
T09	Admin page is not linked in the public navigation	integration
T10–T12	/admin shows login; wrong password rejected; correct password opens panel	integration
T13	All admin actions return 403 without login	integration (security)
T14	Logout removes admin rights	integration (security)
T15–T17	Add location with services; edit location and replace services; delete location incl. service links	integration
T18	Add bilingual FAQ, visible in EN and DE; delete FAQ	integration

ID	Test case (expected result)	Type
T19	Changing contact e-mail/phone updates the footer	integration
T20	Non-numeric coordinates are stored as empty, no crash	integration (robustness)
T21	Location with empty name is rejected	integration (validation)
T22	Contact message with empty fields is rejected by the server	integration (validation)
T23	All images referenced in style.css exist with exactly the same file name	static check (portability)
G01	Map shows 13 markers and 13 list entries	GUI
G02 / G03	Search „bike“ shows 5 bike locations; search by name finds one location	GUI
G04	Search without match shows the „no results“ message	GUI
G05	Search „lap top“ finds the same locations as „laptop“	GUI
G06	Category card „Bikes“ filters to 5 locations; second click resets to 13	GUI
G07	Click on a location shows address and opening hours in the details panel	GUI
G08	Hero search box fills the sidebar search and filters	GUI
G09	DE/EN switch changes the page language	GUI
G10	Contact form submission shows a success message	GUI
G11	Admin logs in, adds a location via the form, location appears on home page	GUI (end-to-end)
G12	No horizontal scrolling at 375 px screen width	GUI (responsive)

Test cases still to be developed: tests for the planned features in the backlog (F16–F19), tests for e-mail delivery once it is implemented, an accessibility check (e.g. with axe-core), load tests before going online, and browser tests in Firefox and Safari (WebKit).

4.2 (E2) Test protocol

The tests were executed by the team on 3 October 2026 on a Windows laptop with Python 3.14.6, pytest 9.1.1 and Playwright (Chromium, headless), using the command python -m pytest tests -v in the project folder. Before the run, the required packages had to be installed with python -m pip, because the Windows security policy on the laptop blocked pip.exe. The complete run took 28 seconds. 35 test cases were executed (plus one helper test that only captures the screenshots for chapter 5): 31 passed and 4 failed. No planned test was skipped. The four failures were left unchanged on purpose and are documented as known errors for the next semester.

Table 12: Test results

ID	Result	ID	Result
T01	Passed	T19	Passed
T02	Passed	T20	Passed
T03	Passed	T21	Failed – a location with the name „ “ was saved (13 → 14 rows)
T04	Passed	T22	Failed – an empty contact message was stored; only the browser (HTML required) checks the fields
T05	Passed	T23	Failed – style.css uses images/audio.jpg and images/tools.jpg, files are Audio.jpg and Tools.jpg
T06	Passed	G01	Passed
T07	Passed	G02	Passed
T08	Passed	G03	Passed
T09	Passed	G04	Passed
T10	Passed	G05	Failed – „lap top“ finds 0 locations, „laptop“ finds 6
T11	Passed	G06	Passed (click dispatched directly, because the cards move in an animated slider)
T12	Passed	G07	Passed
T13	Passed	G08	Passed
T14	Passed	G09	Passed

ID	Result	ID	Result
T15	Passed	G10	Passed
T16	Passed	G11	Passed
T17	Passed	G12	Passed
T18	Passed		

Analysis of the failed tests:

- **T21 / T22 – missing server-side validation:** the admin routes and the contact route save the form values without checking them. Fix: reject empty required fields (and invalid e-mail addresses) in the route functions before writing to the database.
- **T23 – case of file names:** on Windows (development) the file system ignores upper/lower case, so the images are shown; on a Linux server the „Audio & Radio“ and „Tools & DIY“ cards would have no background image. Fix: rename the files to lower case or correct style.css.
- **G05 – search tolerance:** the search compares the text literally. Fix: normalise text before comparing (lower case, remove spaces and hyphens) and add synonyms such as „Handy“/„phone“. This item is already in the backlog for the next semester.

Coverage: the backend tests reach 98 % statement coverage of app.py (194 of 197 statements). Not covered are the error branch for invalid coordinates in the edit route (lines 574–575) and the line app.run() that is only executed when the file is started directly (line 617). The coverage target (≥ 80 %) and the specification coverage target (every necessary requirement has a test) are therefore reached. Additional manual findings: the home page loads about 84 MB of images (N02), and the admin password and secret key are written in app.py (N10).

5 Final report

5.1 (A2) Summary

Over two semesters, the Repair-Help Platform developed from an idea into a working web application. In the winter term we analysed comparable platforms, wrote a requirements specification, built a static prototype with an interactive map and reduced the scope to an information hub after the stakeholder survey. In the summer term we converted the prototype into a Flask application with an SQLite database, added a bilingual interface, live search, category filter, contact form and a password-protected admin panel, integrated 13 real repair locations and applied the new brand identity (logo and brand book). The automated tests show that the main functions work (31 of 35 tests passed, 98 % backend coverage). The platform is not yet online.

Possible future work:

- Preparation for deployment: password and secret key as environment variables, CSRF protection, HTTPS, production WSGI server, e-mail delivery for the contact form, privacy policy and imprint, compressed images (target < 3 MB), pinned Leaflet version, fixing the four failed tests.
- Backlog of the team: spare-parts information from repair cafés; search that also finds similar entries („lap top“ / „laptop“); checking the feasibility of a buy/sell interface and a login for cafés; ideas for building the website similar to eBay; admin access to add or remove the footer data.
- Replace the placeholder links (https://example.com) of the locations with the real websites, and edit FAQs instead of only adding/removing them.
- Align the project name (logo: „Reparaturhilfe“, brand book: „Repair Café Hub Marburg“, website: „Marburg Repair Help“).

5.2 (F1) User documentation

Required dependencies

Python 3.9 or newer and the package Flask (installed from requirements.txt). SQLite is included in Python. A browser with JavaScript and an internet connection are needed for the map.

Installation

```
cd marburg
pip install -r requirements.txt
python app.py
# open http://127.0.0.1:5000/ in the browser
```

On the first start the database file repair_hub.db is created and filled with the 13 locations, 8 services and 3 FAQs. To reset the website to this state, stop the server and delete repair_hub.db. To run the automated tests: pip install pytest playwright coverage, then python -m pytest tests.

Manual for visitors

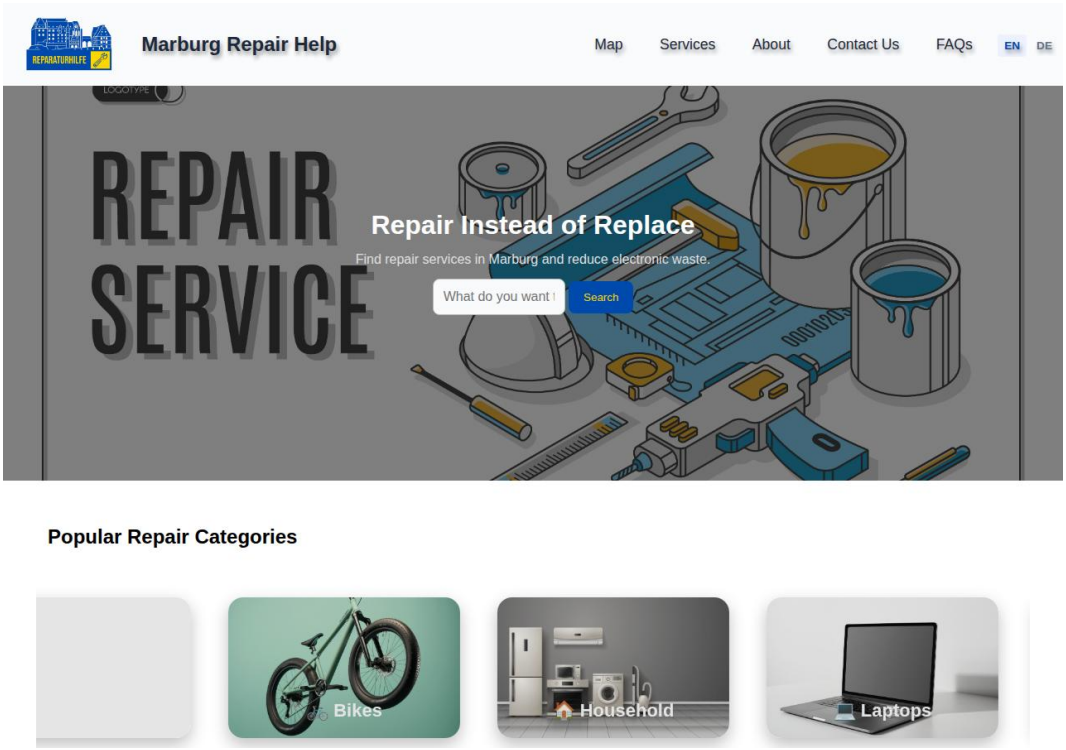


Figure 6: Home page with search box and repair categories

- **Find a location:** scroll to „Repair Locations in Marburg“. All locations are shown on the map and in the list. Type a device or name (e.g. „bike“, „phone“, „HandyDoctor“) into the search field; the list and map update while typing. The search box at the top of the page does the same.
- **Filter by category:** click a category card (e.g. „Bikes“). Click it again to show all locations.
- **See details:** click a location in the list (or a marker). Address, opening hours, services, pricing, accessibility and a link to the provider’s website are shown below the list.
- **Language:** switch between EN and DE in the top right corner.
- **Contact:** use „Contact Us“ to send a message to the team; FAQ and About explain the platform.

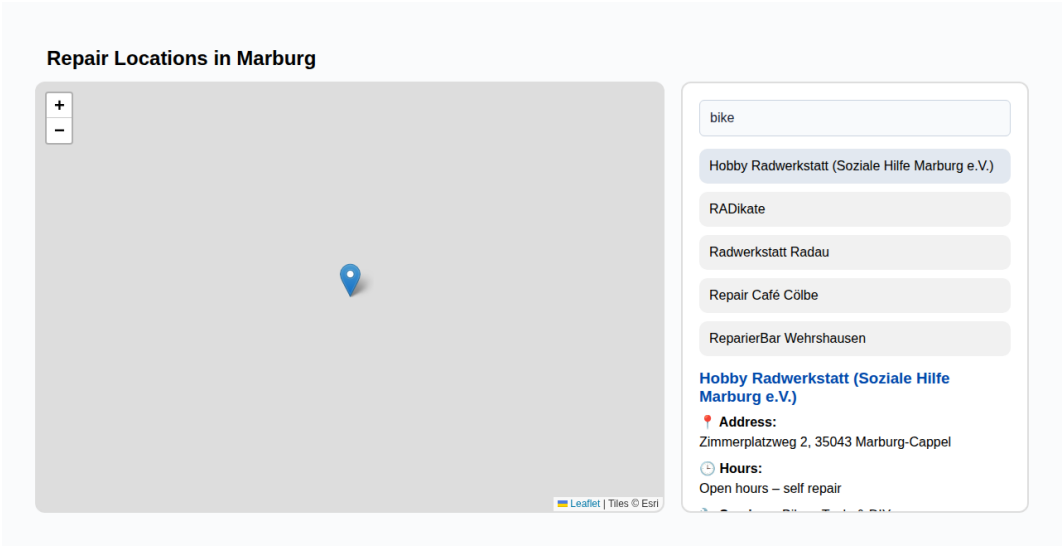


Figure 7: Map and list after searching for „bike“ with the details of a location (map background is grey because external map tiles were disabled in the test environment)

Manual for administrators

Open /admin at the end of the website address (the page is not linked in the menu) and log in with the admin password. The settings page has three sections that open with „Edit“:

- **Contact Email Configuration:** e-mail address and phone number shown in the footer and used for the contact form.
- **FAQ Management:** add a question with answer in English and German, or remove an existing FAQ.
- **Café Management:** add a location (name, address, hours, pricing, accessibility, notes, website, latitude and longitude, offered services), edit an existing one with „Edit“, or delete it with „Remove“. Coordinates can be copied from a map service (e.g. right-click in OpenStreetMap or Google Maps).

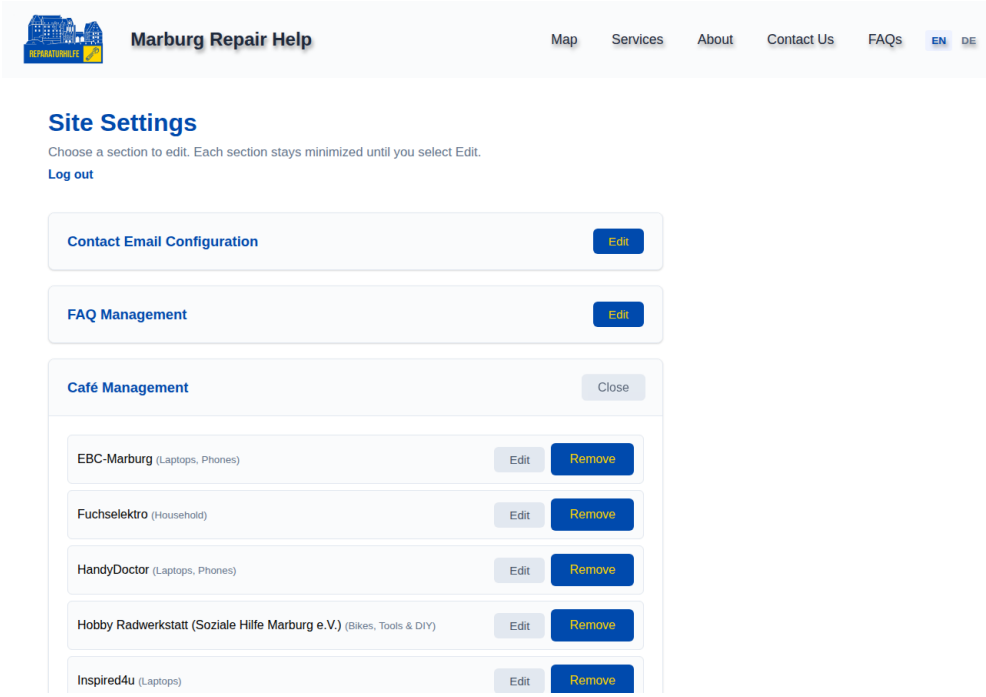


Figure 8: Admin panel with the location management opened

5.3 (F2) Developer documentation

Overview of the implementation

Table 13: Components of the implementation

File / component	Description
app.py	Flask application: configuration (database path, admin password, languages), the i18n dictionary STRINGS (en/de, 94 keys each), database helpers (get_db, init_db, _seed), context processor that passes language, texts and settings to every template, public routes, JSON API and admin routes.
schema.sql	Creates the tables settings, services, cafes, cafe_services, faqs, messages (drops them first).
templates/base.html	Layout shared by all pages: header with logo, navigation and language switch, flash messages, footer with contact data; loads style.css and Leaflet.
templates/index.html	Hero, category slider, CO_2 cards, map and list; embeds the data for script.js as window.__DATA__.
templates/admin*.html	Login form and settings page with collapsible sections and forms for all admin actions.
static/script.js	Map setup (Leaflet), filter state (query, activeService), render(), showDetails(), event handlers.
static/style.css	All styles incl. brand colours and responsive rules (breakpoints 992, 768, 576 px).

File / component	Description
tests/	Automated tests (pytest, Playwright) described in chapter 4.

Expansion scenarios

- **New service category:** insert a row into services (slug, name_en, name_de, icon, featured=1). For a background image, add a CSS class with the same name as the slug in style.css. Currently this needs a database tool or a change of _seed(); an admin form for services would be a useful extension.
- **New interface text:** add the same key to STRINGS['en'] and STRINGS['de'] in app.py and use {{ t.key }} in the template.
- **New language:** add a third dictionary to STRINGS, add the code to SUPPORTED_LANGS and add *_xx columns or a translation table for database texts.
- **New admin function (e.g. footer links, F16):** store the values in the settings table, add a POST route that calls admin_required(), and a form section in admin.html.
- **Fuzzy search (F17):** normalise search text and names in matches() in script.js (lower case, remove spaces/hyphens) and optionally add synonym lists per service.
- **Deployment:** set the secret key and admin password from environment variables, run with Gunicorn behind nginx with HTTPS, and back up repair_hub.db regularly.

Conventions to observe

Keep the separation of route functions, data-access helpers and templates; never write user-visible text directly into templates (use STRINGS); always use SQL parameters (?) instead of string formatting, as in the existing code (protection against SQL injection); redirect after every POST; use service slugs as stable identifiers; follow the brand book for colours and logo use; write a docstring for every new function (guideline, chapter 3) and add tests for every new feature.

Known errors and missing properties

- Search does not find spelling variants (G05).
- No server-side validation of admin and contact form input (T21, T22).
- Image file names differ in case from style.css (T23); images are very large (≈ 84 MB on the home page).
- Admin password („12344“) and secret key are hard-coded; no CSRF protection; debug mode is on when app.py is started directly.
- Contact messages are only stored in the database; no e-mail is sent and there is no admin view for messages.
- Location details are inserted into the page with innerHTML without escaping (only admin input, but should be escaped).
- The elements for the „active filter“ banner used in script.js (filterStatus, clearFilter) do not exist in index.html, so the banner is never shown.
- FAQs cannot be edited, only added/removed; footer and legal links are fixed in base.html; privacy policy and imprint pages do not exist.
- Leaflet is loaded from unpkg without a version number; website links of the locations are placeholders.
- Only 3 of 26 functions in app.py have a docstring (code requirement of the guideline not fulfilled yet).

5.4 (A3) Field report

Decisions that turned out to be good:

- Reducing the scope to an information hub after the stakeholder survey, and managing all content centrally in an admin panel. This avoided building features (accounts for cafés, booking) that nobody would have used.
- Choosing a lightweight stack (Flask, SQLite, Jinja2, plain JavaScript). New members could understand the code within a short time, and the application needs no complex hosting.
- Keeping the existing design when converting the prototype, and building incrementally on it.
- Storing services in a many-to-many relation and texts in both languages from the start.
- Creating a brand book with exact colour codes and accessibility rules, which made the frontend styling straightforward.

Decisions that turned out to be bad:

- Not using a version control system. Versions were copied into separate folders, which made it hard to see what changed and who changed it.
- Adding automated tests only at the end; they found four problems that could have been fixed earlier.
- Working on the brand book in parallel with the frontend, so that colours and names had to be adapted later and the project name is still inconsistent.
- Using the original, uncompressed photos directly as background images.
- Not keeping meeting minutes and not recording the effort per task.

Tips for future developers:

- Start with a Git repository and run the existing tests (python -m pytest tests) after every change; ideally in a CI pipeline.
- Read the README, this report and the brand book before changing the code or design.
- Fix the four failed tests and the security points before publishing the website.
- Agree on requirements with the stakeholders early and check new ideas (e.g. marketplace features) against the decision to build an information platform.
- Use generative AI only as a helper, and always review and test the generated code.